

Lecture 2: Introduction to GitHub and R Basics

Heather Mattie, PhD

September 3, 2026

Recipe of the Day!

Apple cheddar soup



Announcements

- No lab this week
- There will be lab next week
- No TF office hours this week
- Heather's office hour this week is today 2-2:30pm

Office Hours and Lab

Office Hours

Day	Time	Staff	Location
Thursday	12-1 pm	Heather	Building 1, 4 th floor, room 421A
Wednesday	10:30-11:30 am	Stephanie	Building 2, 4 th floor, room 426
Friday	11:30-12:30 pm	Monica	Building 2, 4 th floor, room 426
Monday	1-2 pm	Kimberly	Zoom

Lab

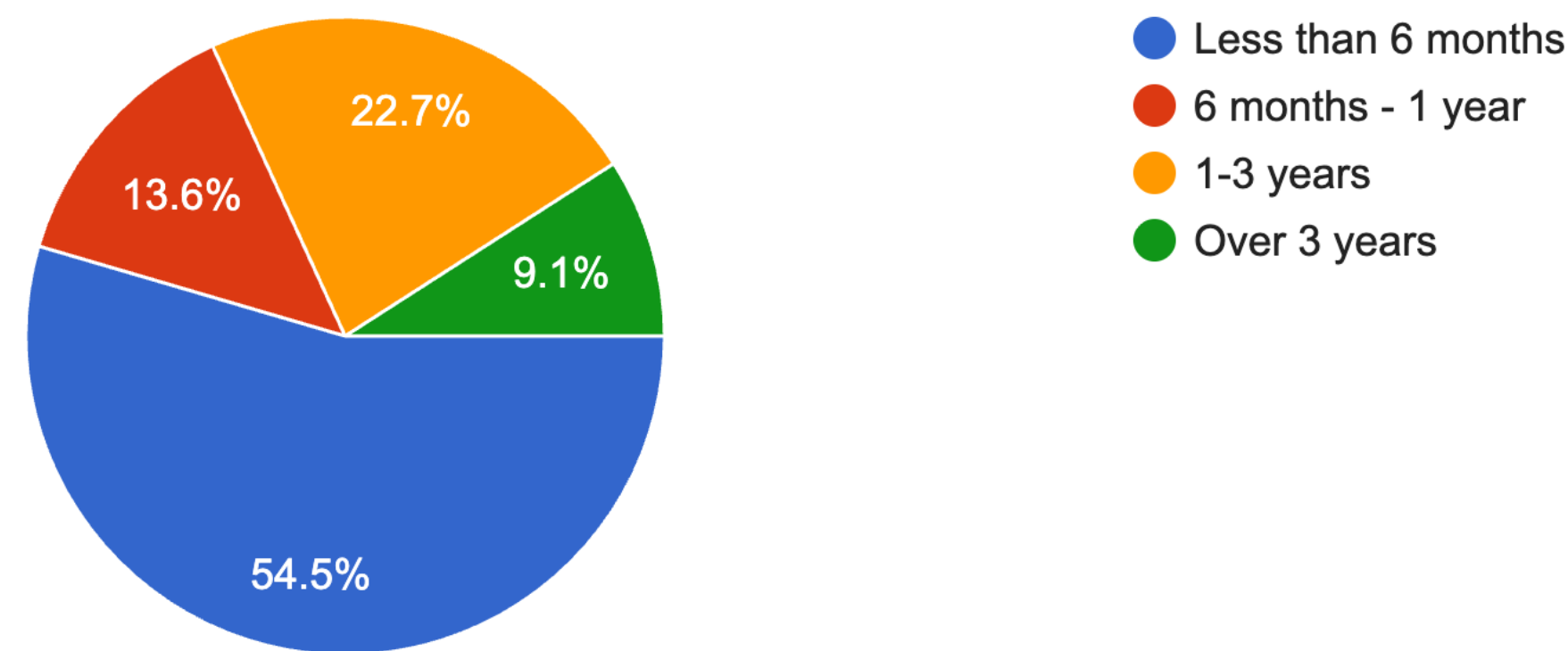
Day	Time	Location
Friday	9:45-11:15 am	Zoom

***Lab will not be held every week**

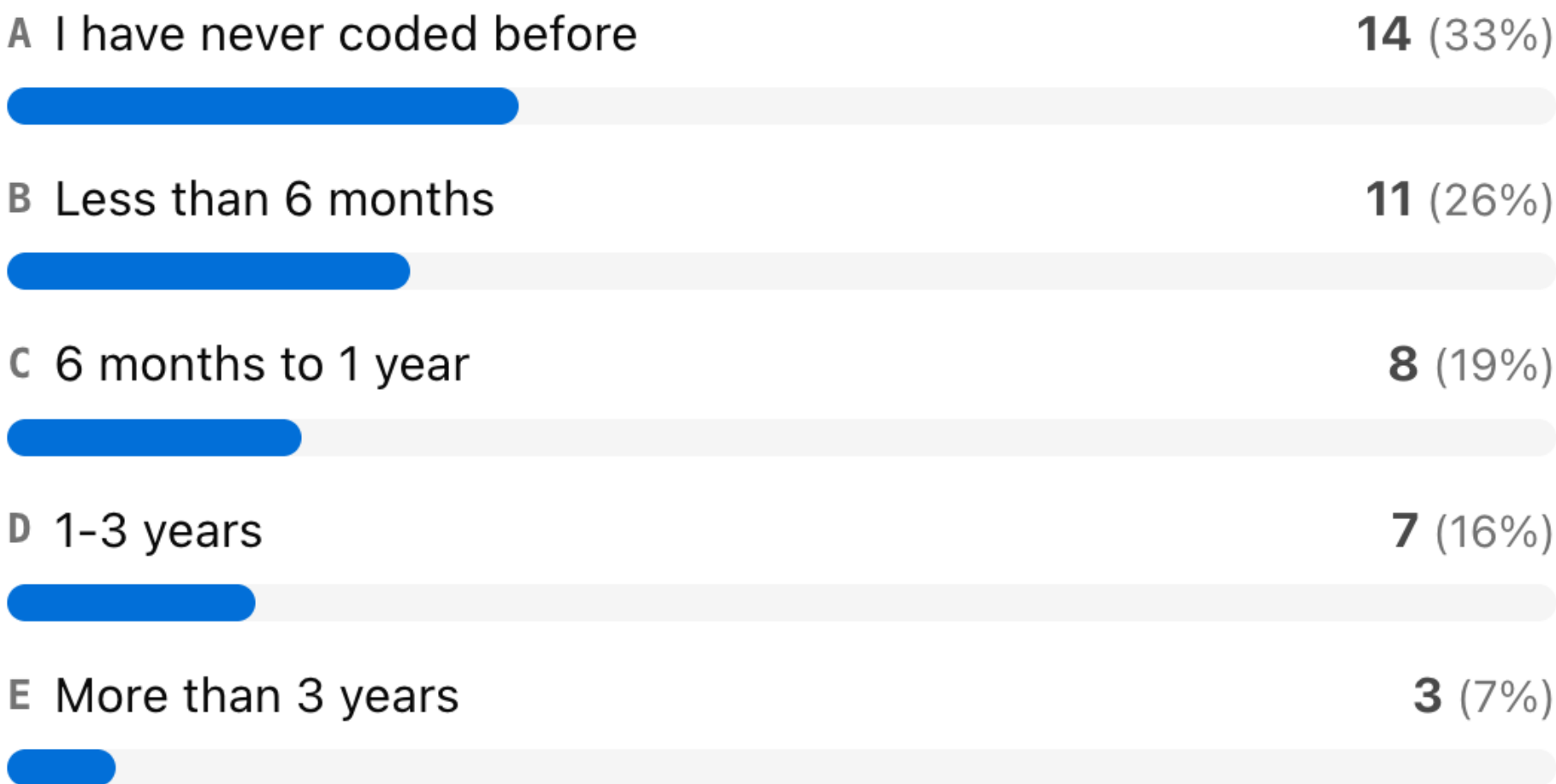


Survey results

How long have you been programming?
22 responses



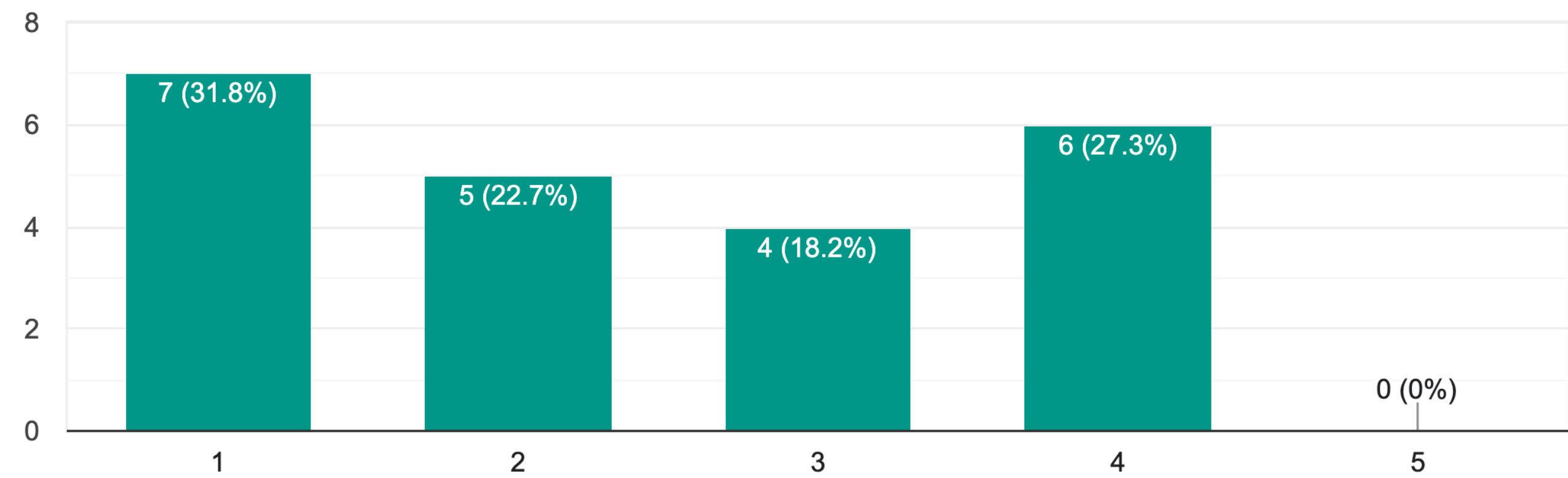
Q2 **MULTIPLE CHOICE** 43 responses
How long have you been coding?



Survey results

Overall, how comfortable are you with programming?

22 responses



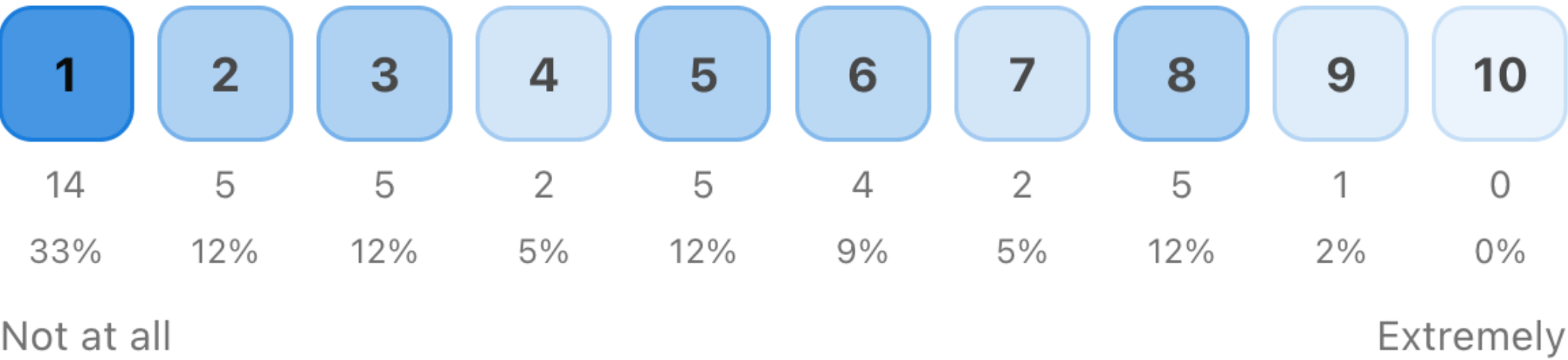
Q3

NUMERIC SCALE

43 responses

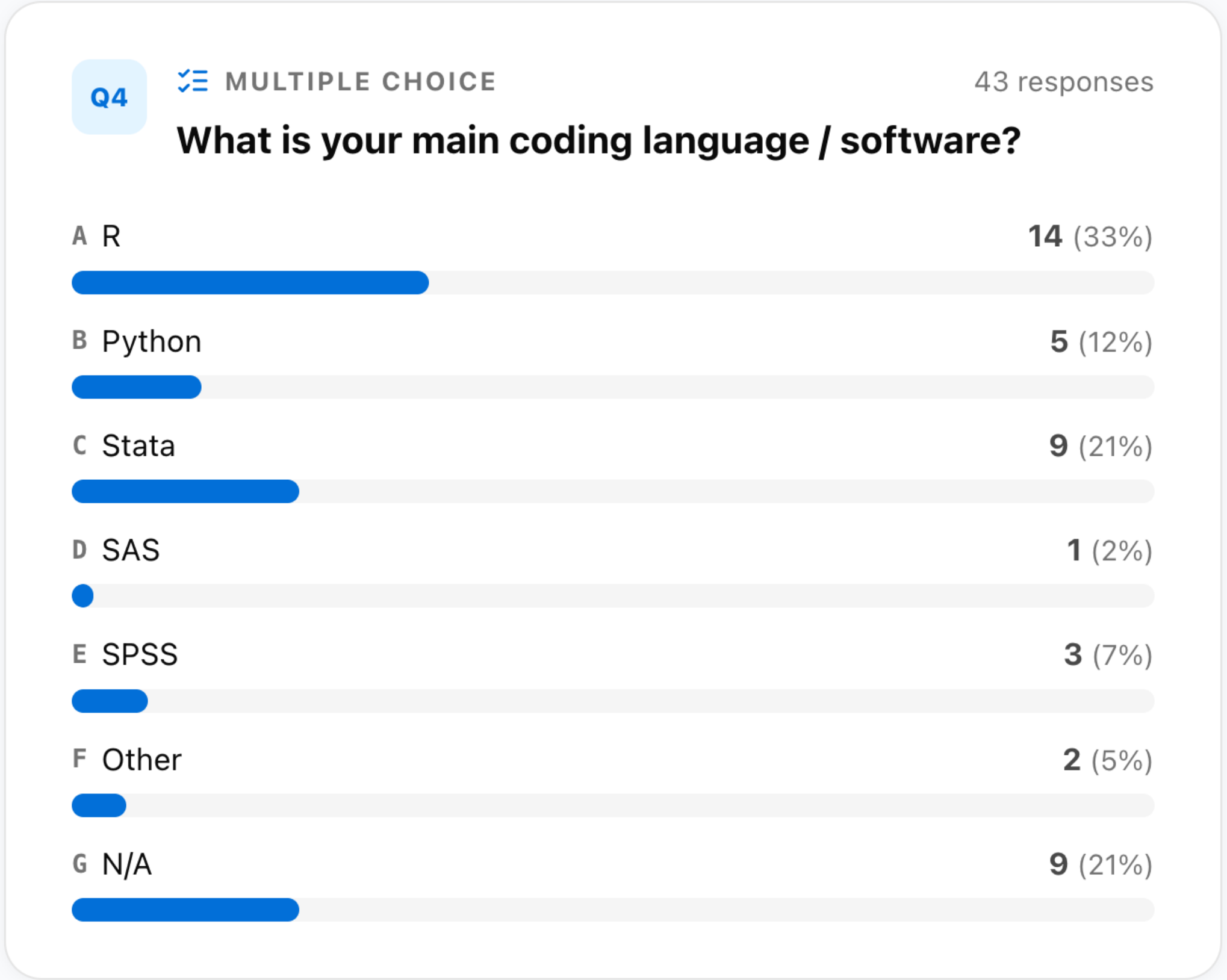
Overall, how comfortable are you with coding?

3.7 avg score



Survey results

- **R proficiency:** A clear majority of respondents identify R as their primary programming language.
- **Stata usage:** Several individuals utilize Stata, with some noting a transition from Stata to R or using both languages for their coursework and research.
- **Additional languages:** A smaller subset of respondents uses Python or possesses a background in C++, while some participants report no prior programming experience or limited exposure through introductory courses.



Survey results

What do you hope to get out of this course?

- **R Programming Proficiency:** A primary goal is to achieve comfort and independence in using R for data cleaning, manipulation, visualization, and efficient processing to support research and clinical analysis.
- **Data Science & Machine Learning:** Participants are seeking to understand statistical methods and acquire specific skills in machine learning to facilitate the analysis of large, complex datasets.
- **Professional Development:** Learners aim to align their coding capabilities with job market demands and acquire formal programming techniques to conduct independent public health and medical research.

To leave this course being able to know when a claim is being made carelessly

Be able to take a data set and use coding to produce informative graphics, statistical analyses, describe trends

Ability to better analyze and visualize program and outcomes data to inform large policy decisions

To get baseline understanding of coding and machine learning. My aim is to create ai scribe software in future.

Learn R, I also work with Claude Code so to be better at understanding code will help me at my job.

To be proficient enough in R to code for clinical research projects (i.e. chart reviews using EMR, making pretty graphs, implementing stat tests like t-tests, p-values on data set)

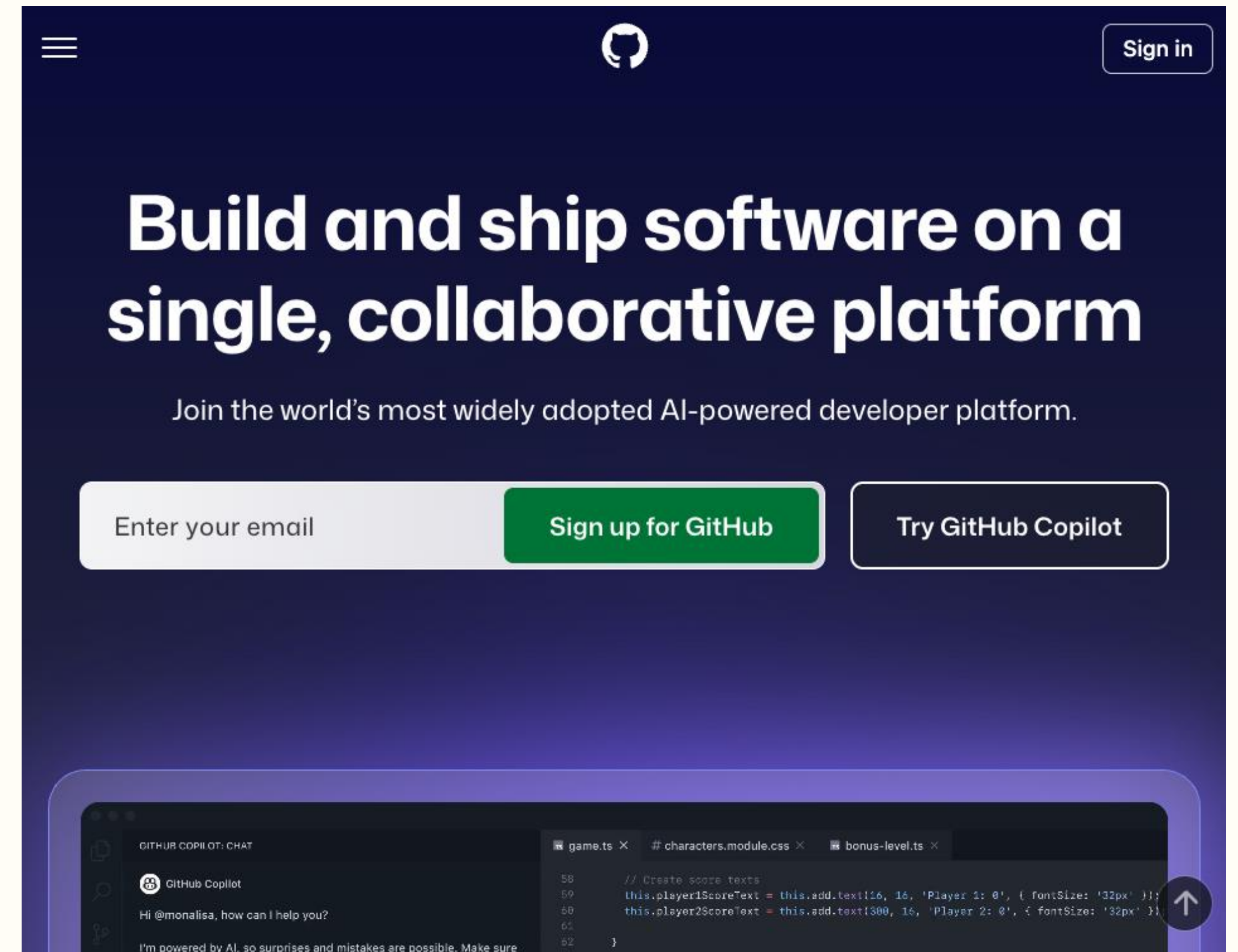
Introduction to GitHub

Heather Mattie, PhD

GitHub

GitHub is a cloud-based platform for software developers and others to build, store, manage, and share code and projects using Git's version control system. It acts as a central hub for collaboration, allowing teams to

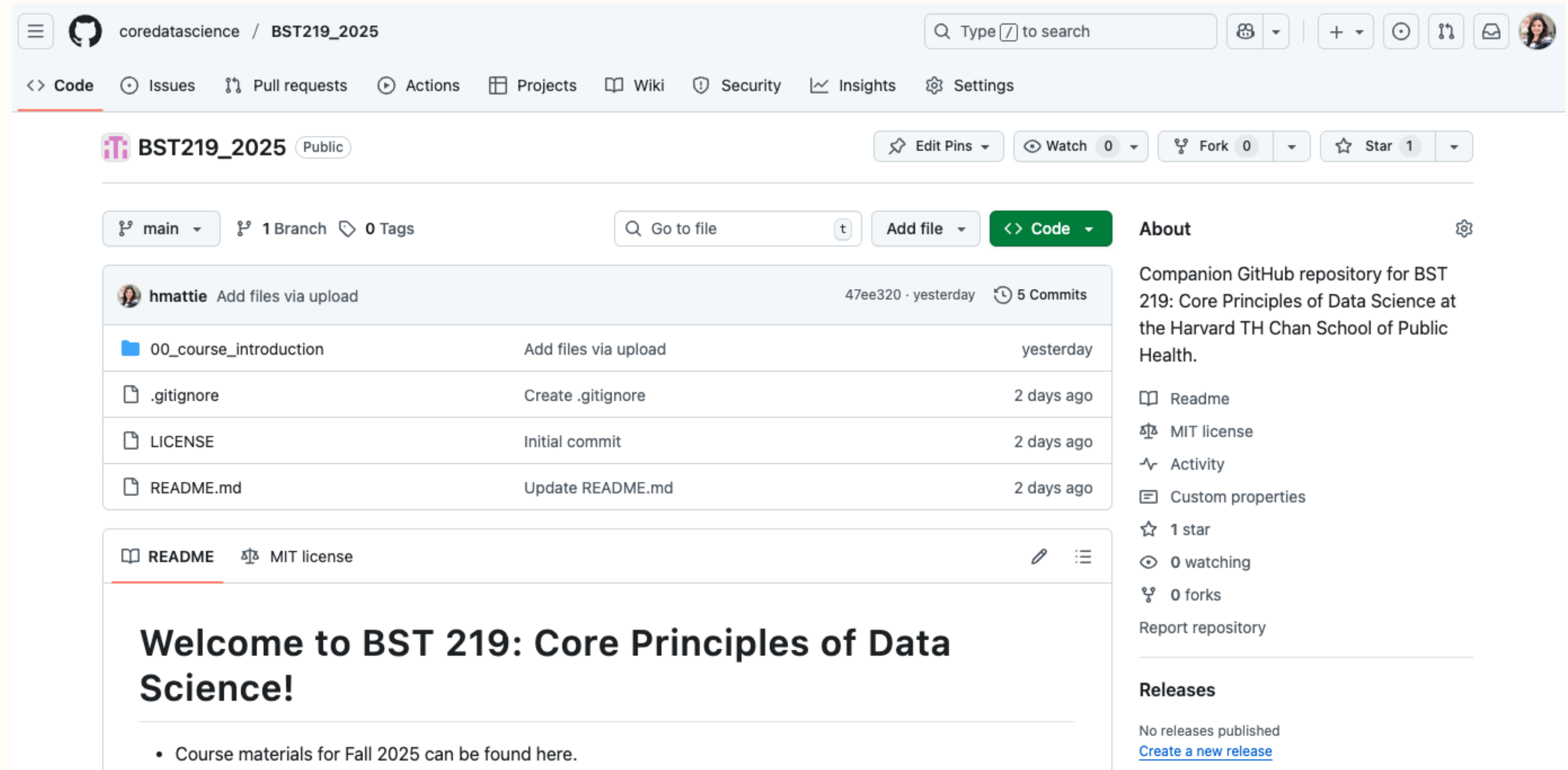
- Track changes
- Work on separate versions at the same time
- Suggest and review contributions
- Manage project tasks



GitHub

What will we use
GitHub for?

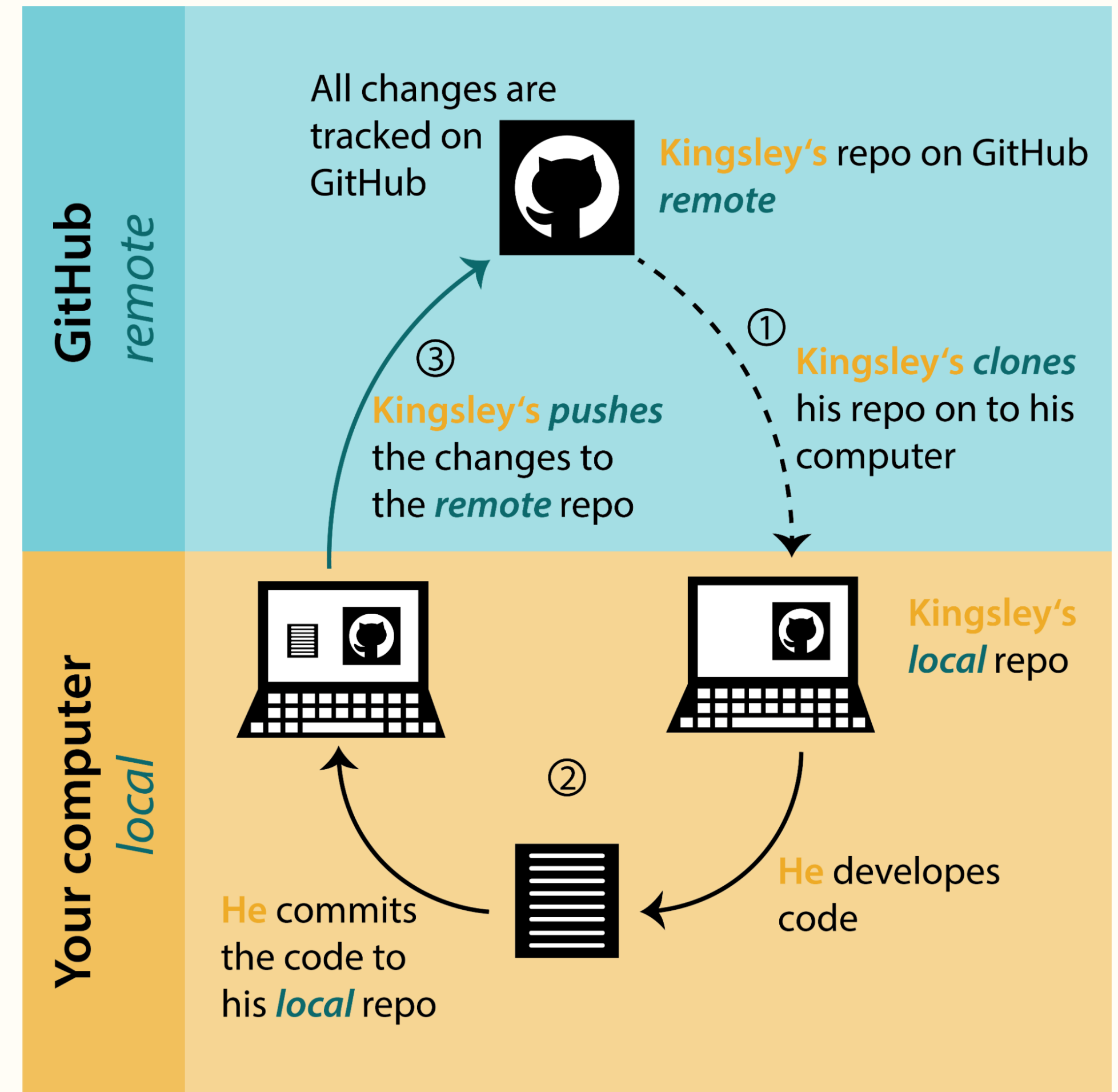
- All course materials



GitHub

The main reasons to use GitHub

- Version control so you don't lose any work
- Always good to try a tool commonly used in data science



GitHub

- GitHub has specific terminology it would be useful to remember.
- Different commands are used for different steps of the version control process. We will only be using a few of them in this course.

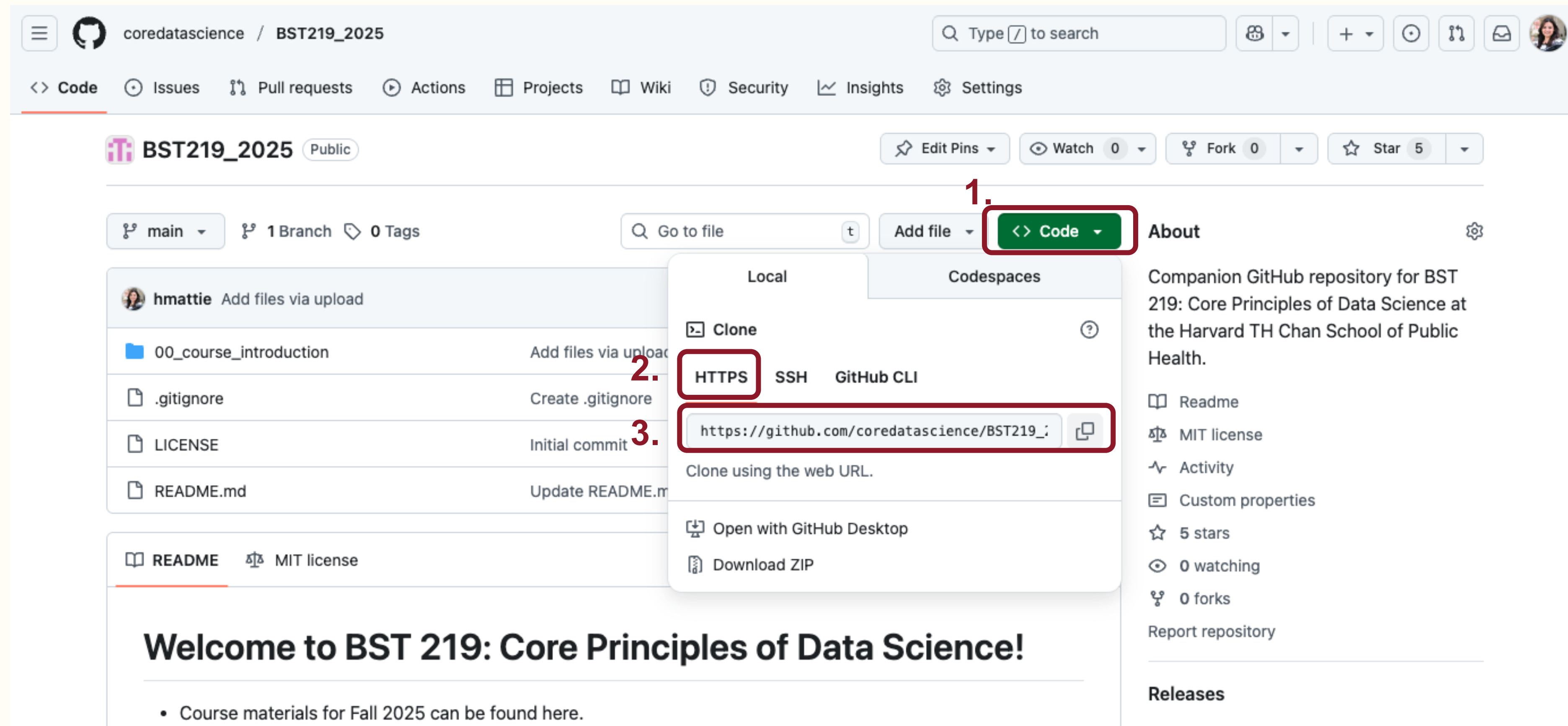
Definitions

- **repo** - short for repository
- **repository** - the directory or folder that is under version control
- **local** - the repository on your computer
- **remote** - the repository on GitHub
- **commit** - take a snapshot of one or more files in the repository
- **push** - send commits from the local repo to the remote repo
- **pull** - retrieve commits from the remote repo to the local repo
- **.gitignore** a file that tells git which files or types of files you don't want to commit

Accessing GitHub in RStudio

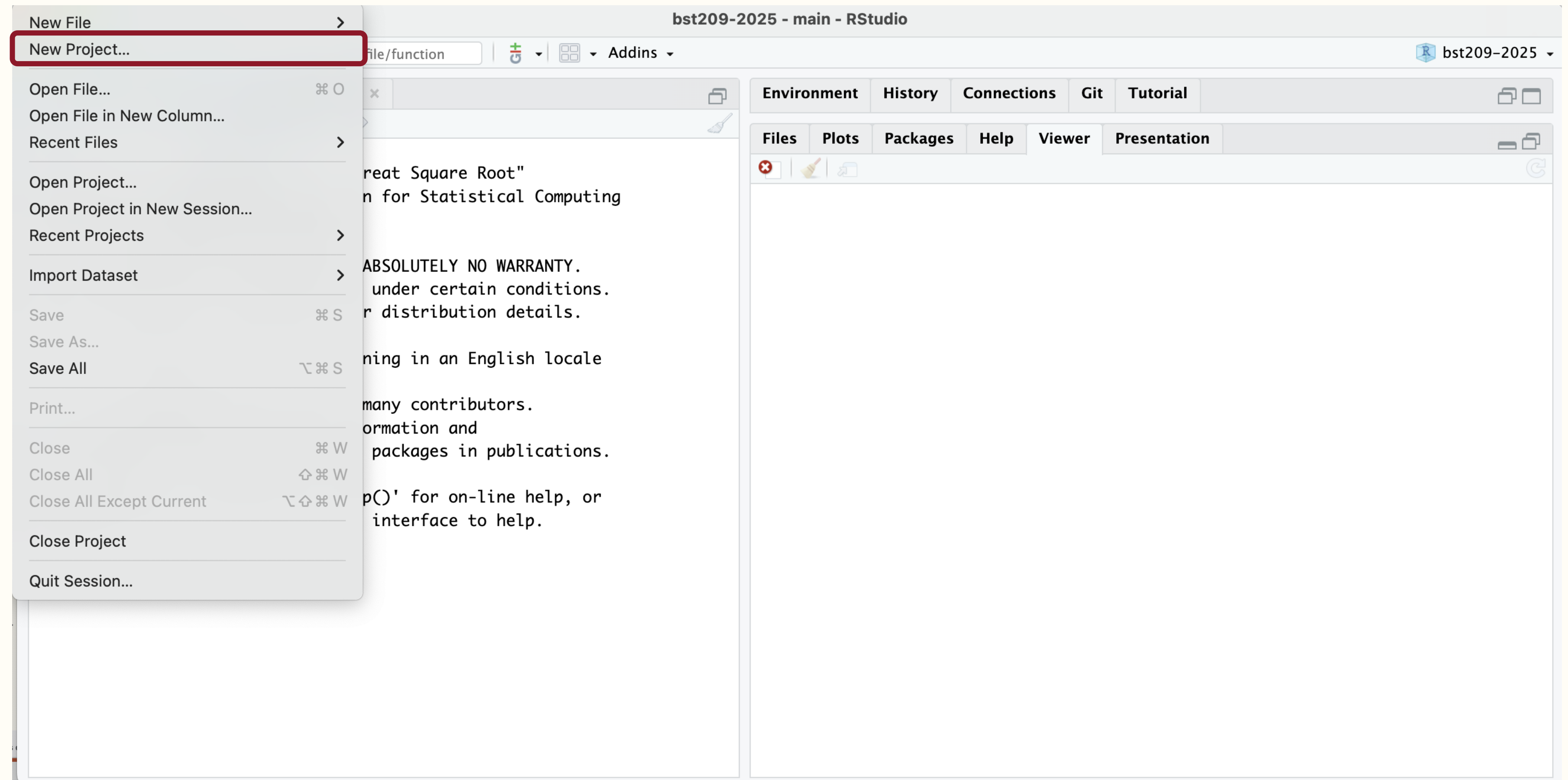
We won't be using the Terminal or Command Line to work with GitHub - we'll be working inside RStudio.

Step 1: Clone the repository



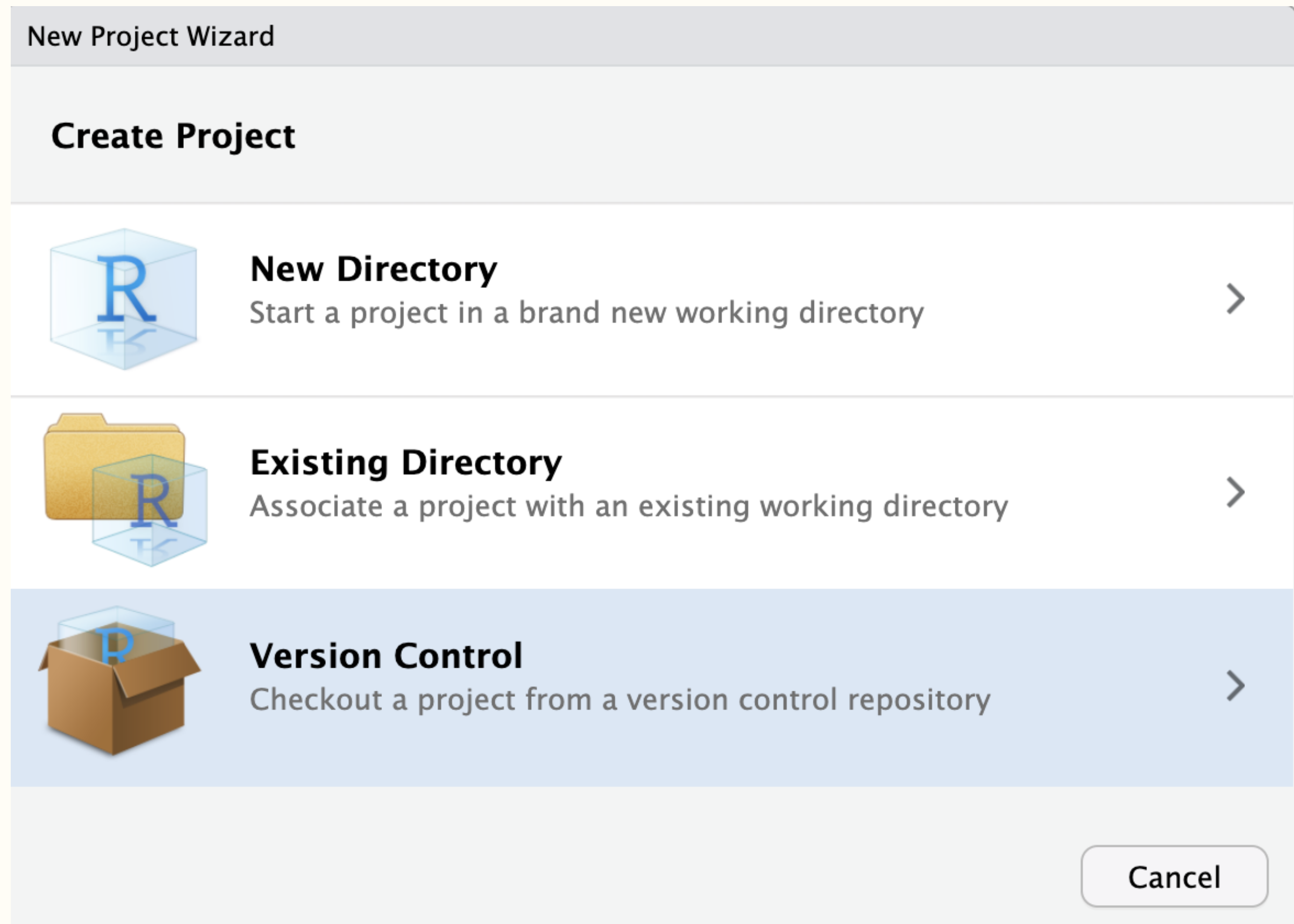
Accessing GitHub in RStudio

Step 2: Open RStudio and select New Project



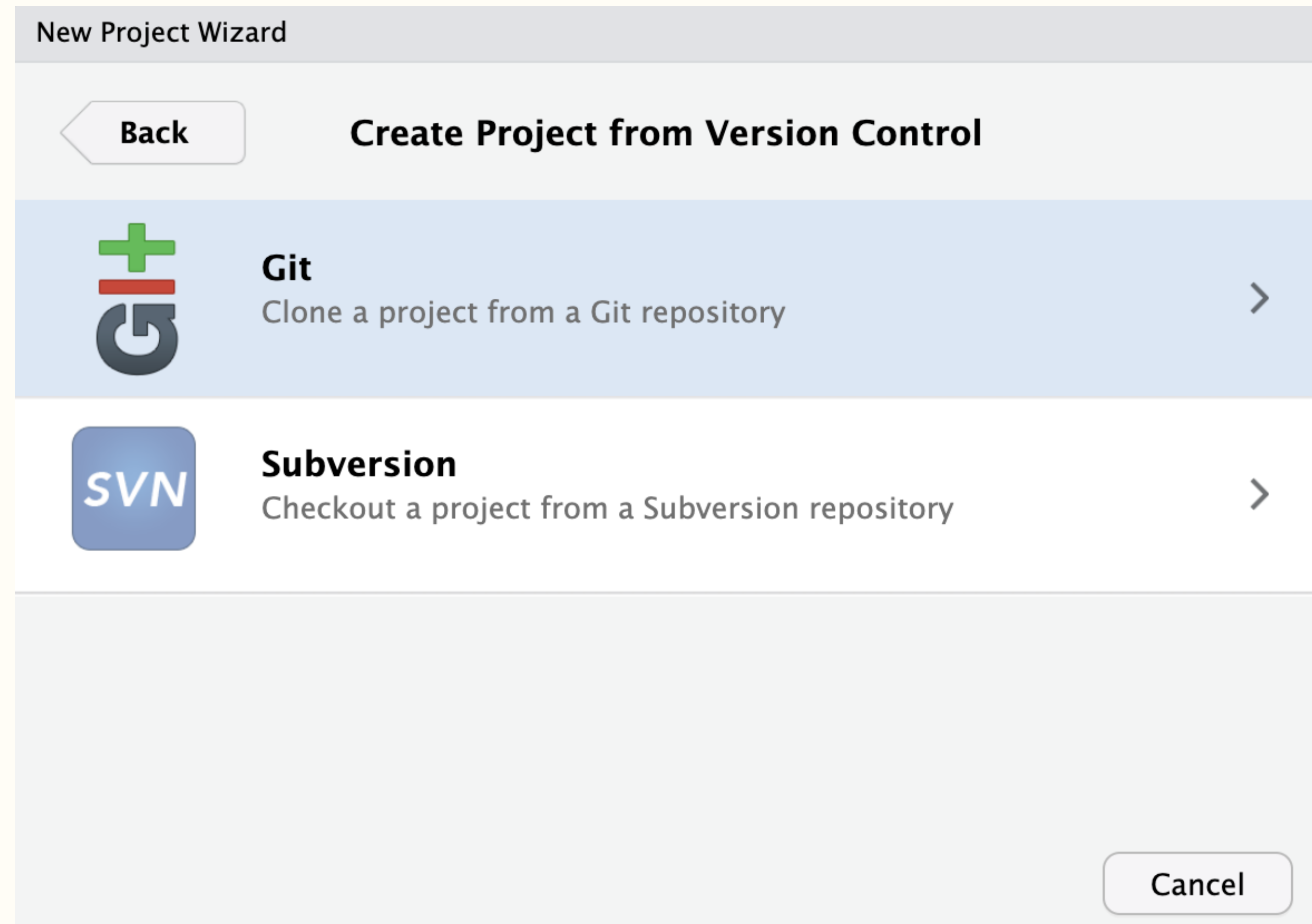
Accessing GitHub in RStudio

Step 3: Select Version Control



Accessing GitHub in RStudio

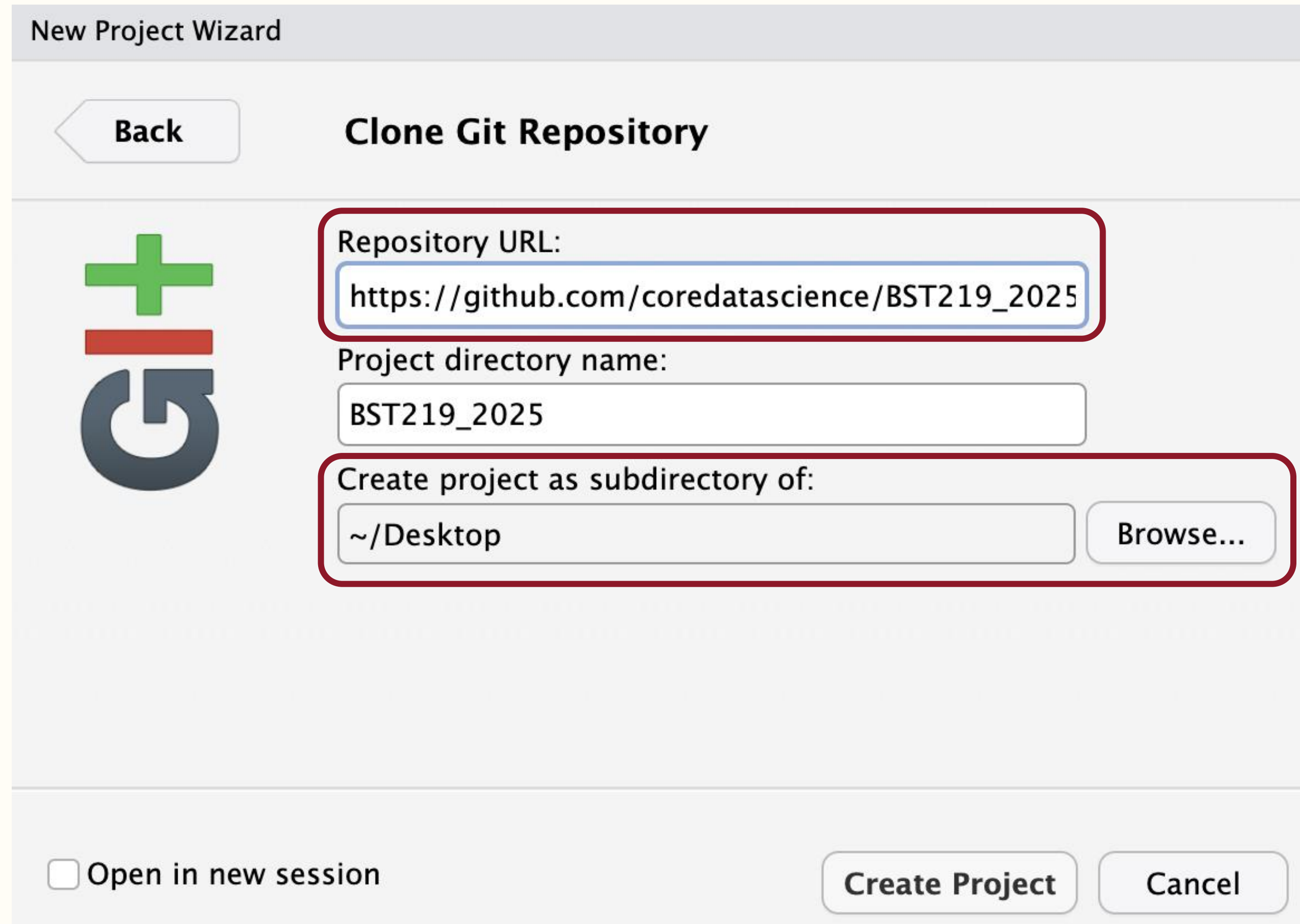
Step 4: Select Git



Accessing GitHub in RStudio

Step 5: Paste the repository link you copied on GitHub

This will become a folder on your computer - take note of where you are saving it!



The screenshot shows the 'New Project Wizard' dialog box in RStudio, specifically the 'Clone Git Repository' step. The dialog has a 'Back' button on the left. On the left side, there is a large green plus sign and a grey 'G' logo. The main area contains three input fields: 'Repository URL:' with the value 'https://github.com/coredatascience/BST219_2025', 'Project directory name:' with the value 'BST219_2025', and 'Create project as subdirectory of:' with the value '~/Desktop'. A 'Browse...' button is next to the last field. At the bottom, there is a checkbox for 'Open in new session' and two buttons: 'Create Project' and 'Cancel'.

New Project Wizard

Back

Clone Git Repository

Repository URL:
https://github.com/coredatascience/BST219_2025

Project directory name:
BST219_2025

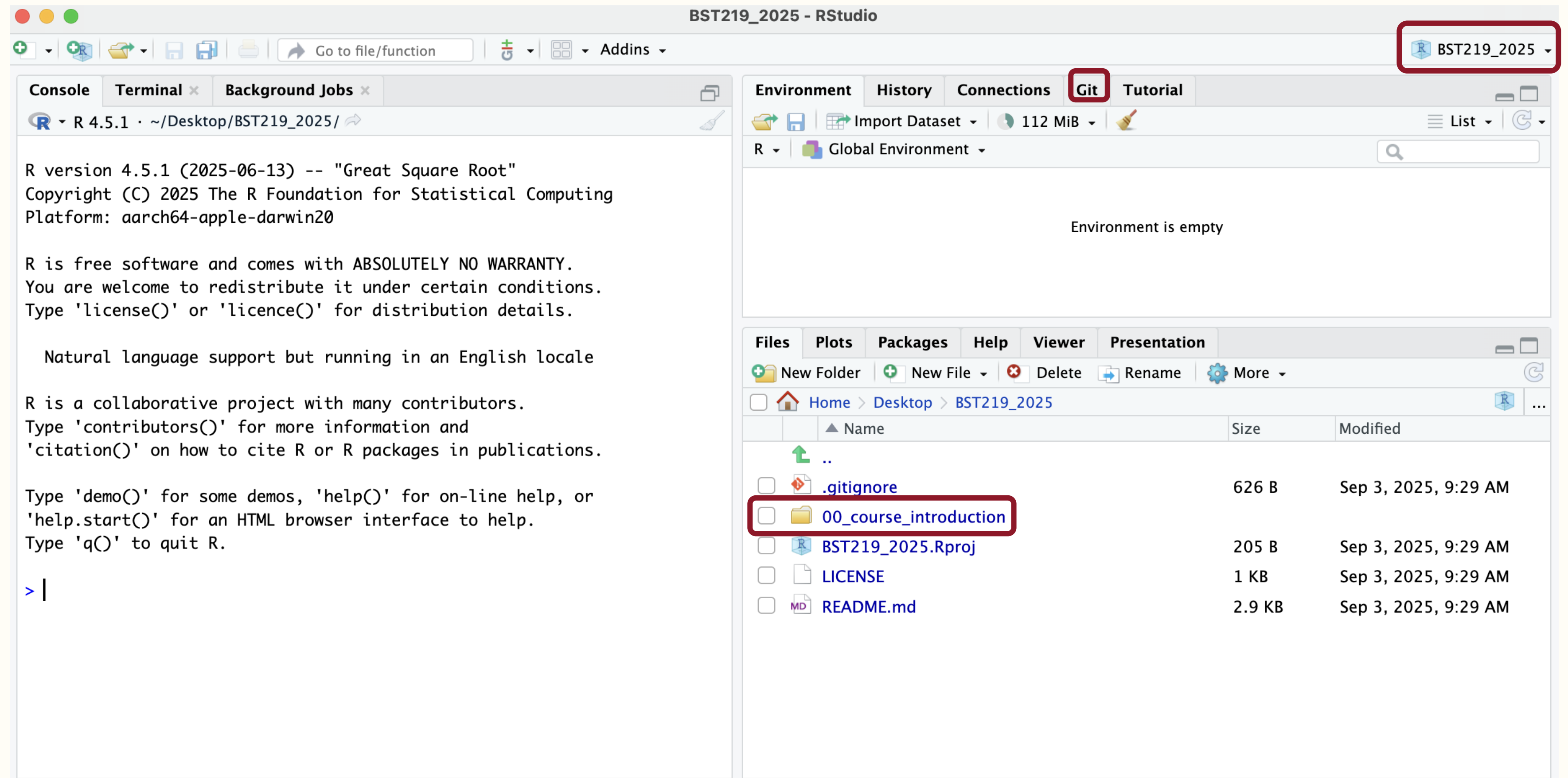
Create project as subdirectory of:
~/Desktop Browse...

☐ Open in new session

Create Project Cancel

Accessing GitHub in RStudio

The project should open and you should see the project (repo) name and all folders and files in the repo.



Saving notes

To write notes/code directly into .Rmd files from the course repository, do the following:

1. Duplicate the file you want to add notes to.
2. Rename the copy of the file the same file name, but add **_personal** at the end.

	01-data-types.html	714.3 KB	Sep 11, 2025, 11:15 AM
	01-data-types.Rmd	8.8 KB	Sep 11, 2025, 11:19 AM
	02-vectors.html	620.5 KB	Sep 11, 2025, 10:53 AM
	02-vectors.Rmd	7.2 KB	Sep 11, 2025, 10:53 AM
	03-sorting_personal.Rmd	6.8 KB	Sep 16, 2025, 10:09 AM
	03-sorting.html	622.5 KB	Sep 16, 2025, 10:58 AM
	03-sorting.Rmd	6.8 KB	Sep 11, 2025, 10:53 AM



R Basics



R Basics Roadmap

1. Data types
2. Vectors
3. Sorting
4. Indexing
5. Basic plots
6. Programming basics

R Basics

Data Types

Heather Mattie, PhD

Data types

A **data type** is a classification that specifies the kind of value a variable can hold and how R should interpret and handle it

- Numeric (integer or decimal)
- Text (string, character)
- True/False value (Boolean, logical)
- Data frame
- List

Data types

Integer

```
x <- 2  
class(x)
```

```
## [1] "numeric"
```

Decimal

```
y <- 3.14  
class(y)
```

```
## [1] "numeric"
```

Text

```
z <- "Boston"  
class(z)
```

```
## [1] "character"
```

Logical

```
a <- TRUE  
class(a)
```

```
## [1] "logical"
```

Data frame

```
library(dslabs)  
data(gapminder)  
d <- gapminder  
class(d)
```

```
## [1] "data.frame"
```

List

```
record <- list(name = "John Doe",  
               student_id = 1234,  
               grades = c(95, 82, 91, 97, 93),  
               final_grade = "A")  
class(record)
```

```
## [1] "list"
```

Examining objects

The function **str** displays more about the structure of an object

```
library(dslabs)
data(gapminder)
str(gapminder)
```

```
## 'data.frame':  10545 obs. of  9 variables:
## $ country      : Factor w/ 185 levels "Albania","Algeria",...: 1 2 3 4 5 6 7 8 9 10 ...
## $ year         : int  1960 1960 1960 1960 1960 1960 1960 1960 1960 1960 ...
## $ infant_mortality: num  115.4 148.2 208 NA 59.9 ...
## $ life_expectancy : num  62.9 47.5 36 63 65.4 ...
## $ fertility     : num  6.19 7.65 7.32 4.43 3.11 4.55 4.82 3.45 2.7 5.57 ...
## $ population    : num  1636054 11124892 5270844 54681 20619075 ...
## $ gdp           : num  NA 1.38e+10 NA NA 1.08e+11 ...
## $ continent     : Factor w/ 5 levels "Africa","Americas",...: 4 1 1 2 2 3 2 5 4 3 ...
## $ region        : Factor w/ 22 levels "Australia and New Zealand",...: 19 11 10 2 15 21 2 1 22 21 ...
```

Examining objects

The function **head** displays the first 6 rows and all columns of a data frame

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1      Albania 1960      115.40           62.87           6.19
## 2      Algeria 1960      148.20           47.50           7.65
## 3       Angola 1960      208.00           35.98           7.32
## 4 Antigua and Barbuda 1960      NA           62.97           4.43
## 5      Argentina 1960      59.87           65.39           3.11
## 6      Armenia 1960      NA           66.86           4.55
## population      gdp continent      region
## 1    1636054      NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844      NA    Africa Middle Africa
## 4      54681      NA Americas Caribbean
## 5    20619075 108322326649 Americas South America
## 6     1867396      NA     Asia Western Asia
```

The accessor

The **accessor operator \$** is used to access different variables.

Most commonly, it is used to access the values in a column in a data frame.

```
library(dslabs)
data(admissions)
names(admissions)
```

```
## [1] "major"      "gender"     "admitted"   "applicants"
```

```
admissions$major
```

```
## [1] "A" "B" "C" "D" "E" "F" "A" "B" "C" "D" "E" "F"
```

Vectors

The `admissions$major` object is not one character, but several – it is a **vector**.

- The number of entries in a vector can be displayed with the **length** function

```
majors <- admissions$major  
length(majors)
```

```
## [1] 12
```

```
class(majors)
```

```
## [1] "character"
```

Factors

Factors are a data type useful for storing categorical data.

R stores these *levels* (categories) as integers but keeps track of the labels by keeping a map of the integer to the level. This is more memory efficient than storing all the characters.

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1         Albania 1960          115.40           62.87         6.19
## 2         Algeria 1960          148.20           47.50         7.65
## 3          Angola 1960          208.00           35.98         7.32
## 4 Antigua and Barbuda 1960             NA           62.97         4.43
## 5         Argentina 1960           59.87           65.39         3.11
## 6          Armenia 1960             NA           66.86         4.55
## population      gdp continent      region
## 1    1636054      NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844      NA    Africa  Middle Africa
## 4      54681      NA Americas    Caribbean
## 5    20619075 108322326649 Americas  South America
## 6    1867396      NA     Asia   Western Asia
```

```
class(gapminder$country)
```

```
## [1] "factor"
```

Summary

- A data type specifies the kind of value a variable can be and how the R should interpret and handle it
- The **str** function displays the structure of an object
- The **head** function displays the first 6 rows of a data frame
- The accessor **\$** is used to access different variables

R Basics

Vectors

Heather Mattie, PhD

Creating vectors

Vectors are created using the concatenate function **c**.

Vectors with text or characters as entries must have quotation marks around each entry.

- Without quotation marks, R will look for variables with those names
- An error will be produced

```
codes <- c(380, 124, 818)
codes
```

```
## [1] 380 124 818
```

```
country <- c("italy", "canada", "egypt")
country
```

```
## [1] "italy" "canada" "egypt"
```

```
country <- c(italy, canada, egypt)
```

Names

It might be useful to **name** the entries of a vector

- Connect the entry with a name for the entry with an equal sign
- Names can be coded with or without quotation marks
- Names can also be assigned using the **names** function

```
codes <- c(italy = 380, canada = 124, egypt = 818)
codes
```

```
## italy canada egypt
## 380 124 818
```

```
names(codes)
```

```
## [1] "italy" "canada" "egypt"
```

```
codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)
codes
```

```
## italy canada egypt
## 380 124 818
```

```
codes <- c(380, 124, 818)
country <- c("italy", "canada", "egypt")
names(codes) <- country
codes
```

```
## italy canada egypt
## 380 124 818
```

Sequences

The **seq** function creates vector sequences

- The first argument defines the start
- The second argument defines the end
- The default increment is to increase by 1 but this can be customized
- There is shorthand for consecutive numbers
- The default class of the entries in a sequence is integer but will be numeric if decimal numbers are part of the sequence

```
seq(1, 10)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(1, 10, 2)
```

```
## [1] 1 3 5 7 9
```

```
1:10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
class(1:10)
```

```
## [1] "integer"
```

```
class(seq(1, 10, 0.5))
```

```
## [1] "numeric"
```

Subsetting

Square brackets are used to access specific elements in a vector

- The position of the entry in the vector, or the name of the entry can be used to access it
- A vector or sequence of numbers can also be used inside the square brackets

```
codes
```

```
## italy canada egypt  
## 380 124 818
```

```
codes[2]
```

```
## canada  
## 124
```

```
codes[c(1,3)]
```

```
## italy egypt  
## 380 818
```

```
codes[1:2]
```

```
## italy canada  
## 380 124
```

```
codes["canada"]
```

```
## canada  
## 124
```

```
codes[c("egypt", "italy")]
```

```
## egypt italy  
## 818 380
```

Coercion

Coercion is R attempting to be flexible with data types

- The entries in a vector must be the same data type
- If an entry isn't the type expected, R will guess what it is and **coerce** the value to be a certain type if it can
- Otherwise, it will give an error
- Numeric entries **can** be coerced into characters
- Character entries **cannot** be coerced into numbers

```
x <- c(1, "canada", 3)
x
```

```
## [1] "1"      "canada" "3"
```

```
class(x)
```

```
## [1] "character"
```

```
x <- 1:5
y <- as.character(x)
y
```

```
## [1] "1" "2" "3" "4" "5"
```

```
as.numeric(y)
```

```
## [1] 1 2 3 4 5
```

Not Availables (NA)

When a coercion function encounters an impossible case, it gives a warning and turns the entry into an **NA**

- NA stands for not available
- NA is also used to denote missing data

```
x <- c("1", "b", "3")  
as.numeric(x)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 1 NA 3
```

Vector arithmetic

Vector arithmetic involves performing mathematical operations on vectors

- Addition, subtraction, multiplication, division, etc.
- In R, arithmetic operations on vectors occur *element wise*

```
heights <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)
```

```
heights * 2.54
```

```
## [1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

```
heights - 69
```

```
## [1] 0 -7 -3 1 1 4 -2 4 -2 1
```

Two vectors

If we have two vectors of the same length, and we sum them, they are added entry by entry

- The same holds for subtraction, multiplication, and division

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Summary

- Vectors are objects with multiple entries that can be created with the concatenate function
- All entries in a vector must be of the same data type
- The entries in a vector can be named
- The **seq** function produces a sequence vector
- Square brackets are used to subset a vector
- R will use coercion if the data types of entries in a vector conflict